

Stop writing loops: your *NumPy* array thinking upgrade

A working ML engineer thinks in arrays, not iterations. Move through every section with your own data — vectorized thinking is a prerequisite for everything downstream: pandas, training loops, tensor ops, inference at scale.

NAME

DATE

SESSION

1 Find your loop problem

- Think of the last time your Python script felt slow — or you reached for a `for` loop to process numbers. Describe it briefly below.
- **What were you computing?** A sum, an average, a transform, a filter? Be specific about the operation, not just the goal.
- **How many rows or values?** Hundreds, thousands, millions? *Scale* changes which fixes matter.
- What happened when you ran it — slow, ugly to read, hard to extend? The pain point is where vectorization pays off.

YOUR LOOP STORY

2 Rewrite one loop as a NumPy operation

- Take a list of numbers you care about — exam scores, expenses, steps walked, stock prices. Put them in an array:

```
import numpy as np
data = np.array([ ... ])          # your real values, here
```

- Write the **loop version** of one calculation (e.g. multiply every value by 1.1). Then rewrite it as a single *vectorized* NumPy expression — `data`

LOOP → VECTORIZED

3 Run your own descriptive stats

- Use your array from section 2. Compute and write the *actual* numbers you got:

```
data.mean()    data.std()    data.min()    data.max()
```

- Now **add one outlier** — a value far outside the rest — and recompute mean and std. How much did each shift?
- What does that tell you about *which stat to trust* when your data has long tails? (Hint: this is why production ML logs medians and

BEFORE & AFTER OUTLIER

4 Build a 2D array from your own data

- Create a **3×4 matrix** from real life — monthly scores, weekly metrics, anything with rows and columns. Then inspect its shape.

```
m = np.array([[.,.,.,.,.],
              [.,.,.,.,.],
              [.,.,.,.,.]])
m.shape      # → (3, 4)      m.ndim      # → 2
```

SHAPE, AXES, MEANING

5 Practice the mental shift

- Finish each sentence in your own words — one line each. This is how the *thinking* upgrade gets locked in.
- **1.** "Before NumPy, when I needed to filter data I would..."
- **2.** "With boolean masking, instead I can..."
- **3.** "The biggest thing that changes when I stop thinking in loops and start thinking in arrays is..."

IN YOUR OWN WORDS